Chapter 6 — Inference Systems Architecture

> *"Training is hard. Inference is where hard becomes expensive. You pay for training once. You pay for inference every second, at every SLA boundary, for the lifetime of the product. Optimize accordingly."* — Production AI Infrastructure Principle

---

Chapter Overview

Training a 70-billion-parameter model is a one-time capital expenditure. Serving it to millions of users is an ongoing operational cost that compounds with every request. The performance engineering decisions made in the inference stack — batching strategy, memory allocation policy, parallelism topology, fleet sizing — determine whether that cost is $0.001 per request or $0.10 per request. A 10× gap in serving efficiency is not hypothetical; it is the difference between profitable and unviable AI products.

Sections in this chapter:

- 6.1 The Two Phases — Prefill and Decode
- 6.2 Continuous Batching — The Foundational Serving Optimization
- 6.3 Chunked Prefill and Its Effect on TTFT P99
- 6.4 Throughput-Latency Tradeoff Curves
- 6.5 TTFT and TPOT — SLA Budget Allocation
- 6.6 vLLM, TensorRT-LLM, and SGLang — Architecture Comparison
- 6.7 Speculative Decoding at the Serving Layer
- 6.8 Disaggregated Prefill and Decode
- 6.9 Multi-GPU Inference — Tensor Parallelism for Serving
- 6.10 Observability Hooks for Production Inference
- 6.11 Hardware Selection for Inference Workloads
- 6.12 Multi-Tenant Serving Cost Model [r1]
- 6.13 The Tokens-Per-Second Optimization Hierarchy [r1]
- 6.14 Key Takeaways and Review Questions

6.1 The Two Phases — Prefill and Decode

6.1.1 Why They Are Fundamentally Different Operations

LLM inference is not a single operation. It consists of two phases with different compute profiles, different bottlenecks, and — in modern production systems — different optimal hardware. Understanding this split is the single most important concept in LLM inference engineering. Every optimization in this chapter exists because of the consequences of this dichotomy.

Prefill: Process the entire input prompt in a single parallel forward pass. All N prompt tokens flow through every transformer layer simultaneously. This produces a large, dense matrix multiply — the effective "batch size" for the GEMM is N (the number of prompt tokens). It is compute-bound.

Decode: Generate one new token at a time, autoregressively. Each step processes exactly one token. The effective GEMM is a matrix-vector multiply (GEMV). Every weight byte in the model must be loaded from HBM for a single FLOP of useful computation. It is memory-bandwidth-bound.

```
ROOFLINE CLASSIFICATION — PREFILL vs DECODE:

  H100 SXM5:  989 TFLOPS BF16, 3.35 TB/s HBM, ridge point ~295 FLOP/byte

  PREFILL (LLaMA-3 70B, N=2048 prompt tokens, BF16):
    FLOPs per forward pass:  2 × 70B × 2048 = ~287 TFLOPS
    Weight bytes loaded:     ~140 GB (70B × 2 bytes/param)
    Arithmetic Intensity:    287 × 10^12 / 140 × 10^9 = ~2,050 FLOP/byte
    Status:  AI >> 295 → COMPUTE-BOUND ✓
    Optimization lever: reduce FLOPs (FlashAttention, FP8, pruning)
    Bottleneck: Tensor Core throughput

  DECODE (LLaMA-3 70B, batch=1, BF16):
    FLOPs per decode step:   2 × 70B × 1 = ~140 GFLOPs
    Weight bytes per step:   ~140 GB (full model loaded each step)
    Arithmetic Intensity:    140 × 10^9 / 140 × 10^9 = 1.0 FLOP/byte
    Status:  AI << 295 → MEMORY-BANDWIDTH-BOUND ✓
    Optimization lever: increase batch size, reduce bytes (quantization)
    Bottleneck: HBM bandwidth

  CRITICAL IMPLICATION:
    Adding Tensor Core TFLOPS does NOT improve single-request decode latency.
    H100 is faster for decode than A100 only because HBM bandwidth is higher
    (3.35 vs 2.0 TB/s) — not because of compute improvements.

    Maximum decode throughput at batch=1 (bandwidth-limited):
      tokens/sec = HBM_bandwidth / model_bytes
                 = 3,350 GB/s / 140 GB = ~24 tok/s (theoretical)
                 × 0.70 utilization = ~17 tok/s (practical)

    At batch=128, decode becomes near-compute-bound:
      AI = 2 × 128 = 256 FLOP/byte (approaching 295 ridge point)
      Quantization and batch size are the two primary decode throughput levers.
```

6.1.2 TTFT vs. TPOT — What Each Measures

```
TTFT — TIME TO FIRST TOKEN:
  Definition:  Wall-clock time from request arrival to first output token
               emitted to the client
  User experience: "Is it responding?" Initial wait perception.
  Driven by:
    1. Prompt length N (more tokens -> more prefill compute -> longer TTFT)
```

```
    2. Request queue depth (waiting for a free GPU batch slot)
    3. Current batch occupancy (prefill interleaved with other work)
    4. Chunked prefill policy (affects per-step prefill cost)

  Formula:
    TTFT = t_queue + t_prefill + t_first_decode
    t_prefill ≈ (2 × N × FLOPs_per_layer × L) / (peak_TFLOPS × MFU)
    For N=512, L=80, d_model=8192, H100:
      FLOPs = 2 × 512 × 2×8192²×80 ≈ 4.4 T
      t_prefill ≈ 4.4T / (989T × 0.55) = ~8 ms (GPU time only)

TPOT — TIME PER OUTPUT TOKEN:
  Definition:  Time between consecutive generated tokens after the first
  User experience: "How fast is it typing?" Perceived generation speed.
  Driven by:
    1. Model size (larger model = more bytes to load per decode step)
    2. Batch size (higher batch = more AI = faster per-token amortized time)
    3. KV cache length (longer context = more attention DRAM traffic)
    4. Hardware HBM bandwidth (the fundamental ceiling at small batch)

  At batch=1, BF16:
    TPOT ≈ model_bytes / HBM_bandwidth = 140 GB / 3350 GB/s = 42 ms/token
    = ~24 tokens/second  (maximum, ignoring attention overhead)

  At batch=32, BF16 (practical serving):
    TPOT ≈ (model_bytes + KV_per_token × seq_len) / HBM_bandwidth × (1/batch)
         ≈ 15-25 ms/token (amortized across batch)

SLA RELATIONSHIP:
  TTFT SLA and TPOT SLA are independent but interact:
    Large batch -> better TPOT (amortized BW) but worse TTFT (queue time)
    Small batch -> better TTFT but worse TPOT and utilization
    Chunked prefill resolves the conflict (see §6.3)

  Target ranges (production chatbot serving):
    TTFT P50: < 300ms   |  TTFT P99: < 800ms
    TPOT P50: < 40ms    |  TPOT P99: < 80ms
```

6.2 Continuous Batching — The Foundational Serving Optimization

6.2.1 The Problem with Static Batching

Before continuous batching (2022), LLM serving used static batching: group N requests, run the full batch until ALL requests complete, then start the next batch. This is catastrophically inefficient for variable-length outputs.

```
STATIC BATCHING — THE STRAGGLER PROBLEM:

  Batch: [Req A: 20 output tokens, Req B: 200 output tokens, Req C: 50 tokens]
  GPU executes 200 decode steps for the full batch.

  Steps 1-20:   Run [A, B, C]
  Steps 21-50:  Run [padding, B, C]   <- A finished, GPU wastes cycles on its slot
  Steps 51-200: Run [padding, B, padding]  <- C finished, B finishes at step 200

  GPU utilization: A and C's slots idle for 90% and 75% of decode time.
  Overall GPU idle: ~50-70% of decode time wasted on padding/straggling.

  Waiting requests: queued until the ENTIRE batch finishes.
  A new 10-token request must wait 200 decode steps even if 190 slots are idle.

THROUGHPUT IMPACT:
  Static batching effective throughput: 20-50% of theoretical maximum.
  The longer the tail of the output distribution, the worse this gets.
```

6.2.2 Continuous Batching — Iteration-Level Scheduling

Continuous batching (Orca, 2022; vLLM, 2023) inserts new requests at the iteration boundary — the moment any existing sequence completes a decode step.

```
CONTINUOUS BATCHING EXECUTION:

  Step 1:  Batch = [A, B, C]
  Step 21: A finishes! Immediately admit D (was waiting in queue).
           Batch = [D, B, C]    <- zero idle cycles between A ending and D
starting
  Step 51: C finishes! Admit E.
           Batch = [D, B, E]
  ...

  GPU utilization: maintained near peak throughout.
  Waiting requests: admitted as fast as slots open, not at batch boundaries.

THROUGHPUT IMPROVEMENT:
  Static batching:      GPU idle 50-70% of decode time
  Continuous batching:  GPU idle <5% of decode time
  Throughput gain:      2-4× (varies by output length distribution variability)

  At equal TTFT target: continuous batching serves 2-4× more users on same
hardware.
  At equal hardware: continuous batching reduces TTFT by reducing queue time.

  This is the single highest-ROI optimization in LLM serving.
  Enable it always. There is no scenario where static batching is preferable.

IMPLEMENTATION CONSTRAINTS:
  Requires iteration-level scheduler (not batch-level).
  Must manage KV cache allocation dynamically as sequences enter/exit.
  Requires preemption support: if KV cache fills, can pause a request.
  vLLM, TRT-LLM, SGLang all implement this natively.

KEY PARAMETER: max_num_seqs
  Controls maximum concurrent sequences in a batch.
  Higher -> better GPU utilization, higher TPOT variance.
  Lower -> lower TTFT, lower throughput.
  Recommended starting point: max_num_seqs = 256 for H100 (LLaMA-3 70B FP8)
```

6.3 Chunked Prefill and Its Effect on TTFT P99

Continuous batching maximizes throughput but creates a new TTFT problem: a single long prefill monopolizes the GPU for tens of milliseconds, blocking all decode steps (and all

waiting requests) during that time.

6.3.1 The Long-Prefill TTFT Spike

```
LONG PREFILL BLOCKING PROBLEM:

  Scenario: Requests in queue
    Req X:  8,192-token prompt (long document analysis)
    Req Y:  short 64-token request (waiting behind X)

  Without chunked prefill:
    GPU runs Req X full prefill: ~150ms (8K tokens, compute-bound)
    During this 150ms:
      All decode steps are blocked (X is monopolizing the GPU)
      Req Y is blocked from starting
    Req Y's TTFT = 150ms queue + its own prefill

    If SLA is P99 TTFT < 300ms, a single 8K-token request in the batch can
    push ALL other requests' TTFT above 300ms during its prefill phase.

  P99 TTFT spike pattern:
    P50 TTFT: 120ms (normal traffic)
    P99 TTFT: 1,200ms (when a 8K+ prompt lands in the batch)
    P99/P50 ratio >> 3x -> prefill monopoly is the cause
```

6.3.2 Chunked Prefill Mechanics

```
CHUNKED PREFILL SOLUTION:

  Instead of running full prefill of a long prompt in one GPU step,
  break it into fixed-size chunks and interleave with decode steps.

  Config: chunk_size = 512 tokens

  Step 1:  Process Req X tokens 0-511   (prefill chunk 1) + 32 decode seqs
  Step 2:  Process Req X tokens 512-1023 (prefill chunk 2) + 32 decode seqs
  ...
  Step 16: Process Req X tokens 7680-8192 (prefill chunk 16) + 32 decode seqs
           -> Req X TTFT delivered after step 16 (no monopoly at any step)
           -> All decode steps continue uninterrupted
           -> Req Y never blocked more than one chunk period (~8ms for 512 tokens)

PARAMETER TRADEOFFS:
```

```
  chunk_size = 512:   Low TTFT spike (max 8ms monopoly per chunk),
                      slight throughput overhead (scheduling per chunk)
  chunk_size = 1024:  Moderate TTFT spike (max 16ms), less scheduling overhead
  chunk_size = 2048:  Higher TTFT spike (max 30ms), minimum overhead
  chunk_size = None:  No chunking (full prefill, high TTFT P99 risk)

WHEN TO ENABLE CHUNKED PREFILL:
  Enable when:
    P99 prompt length > 1,024 tokens
    TTFT P99 SLA < 500ms
    Serving interactive workloads (chat, agents with tool calls)

  Consider disabling when:
    All prompts are < 256 tokens (overhead without benefit)
    Batch API workloads where TTFT matters less than throughput
    Pure batch processing (no SLA on TTFT)

vLLM CONFIGURATION:
  # Enable chunked prefill
  --enable-chunked-prefill
  --max-num-batched-tokens 512     # chunk size
  --max-num-seqs 256               # max concurrent sequences

TTFT IMPROVEMENT WITH CHUNKED PREFILL:
  Without: TTFT P99 = 800ms (long-prompt spikes)
  With:    TTFT P99 = 220ms (bounded by chunk size + queue time)
  Throughput cost: ~3-8% (more scheduling iterations)
  Net verdict: almost always worth it for interactive workloads.
```

6.4 Throughput-Latency Tradeoff Curves

Every LLM serving system faces a fundamental tension between throughput and latency. Understanding this curve is essential for capacity planning and SLA design.

```
THROUGHPUT-LATENCY CURVE — LLaMA-3 70B, H100×8 (TP=8), vLLM, FP8:

  Operating  Throughput  TTFT P50  TTFT P99  GPU Util  Notes
  Point      (tok/s)     (ms)      (ms)       (%)
  ──────────────────────────────────────────────────────────────
  A: 10 RPS  12,400      85ms      210ms      15%      Under-utilized, expensive
  B: 30 RPS  31,200      95ms      280ms      40%      Warm, efficient
  C: 60 RPS  52,800      120ms     420ms      70%      Sweet spot
  D: 80 RPS  61,600      185ms     680ms      82%      Approaching SLA limit
  E: 95 RPS  63,200      380ms     1,800ms    88%      SLA violation zone
  F: 100 RPS 62,400      1,200ms   6,000ms    90%      Collapse (queue grows)
  ──────────────────────────────────────────────────────────────


  THE KNEE OF THE CURVE:
    Operating point C-D (70-82% GPU utilization) is the optimal operating region.
    Below: hardware is wasted (high cost per token).
    Above: latency degrades non-linearly (queueing theory: M/M/1 instability).

  LITTLE'S LAW APPLIED:
    L = λ × W
    L = avg requests in system (queue + being served)
    λ = arrival rate (req/s)
    W = avg time in system (queue wait + inference time)

    At 90% utilization (M/M/1 approximation):
      Mean queue depth ≈ 9 waiting requests
      P99 TTFT ≈ 9× service time -> latency collapse

    At 70% utilization:
      Mean queue depth ≈ 2.3
      P99 TTFT ≈ 2.1× service time -> controllable

PRODUCTION FLEET OPERATING TARGETS:
  Interactive serving:  65-75% average GPU utilization
  Batch API:            75-85% average GPU utilization
  Maximum safe:         82% (beyond this, P99 TTFT violates most SLAs)
  Never run at:         > 85% sustained (queue grows faster than it drains)
```

6.5 TTFT and TPOT — SLA Budget Allocation

6.5.1 Decomposing TTFT into Budget Components

```
TTFT LATENCY DECOMPOSITION:

  TTFT = t_network + t_auth + t_queue + t_prefill + t_first_decode_step

  Component    Typical Value  P99 Multiple  Primary Lever
  ──────────────────────────────────────────────────────────────
  t_network    5-30ms         2×            CDN placement
  t_auth       5-20ms         3×            Cache auth tokens
  t_queue      0-500ms+       10×+          Fleet sizing, admission ctrl
  t_prefill    8-150ms        5×            Chunked prefill, FP8
  t_first_tok  15-45ms        1.5×          TPOT optimization
  ──────────────────────────────────────────────────────────────

  KEY INSIGHT: t_queue is the most variable component and the most
  commonly under-estimated. At 80%+ GPU utilization, queue wait can
  dominate TTFT P99, turning what looks like a kernel/hardware problem
  into a fleet sizing problem.

  P99 TTFT SPIKE PATTERN CLASSIFICATION:

  Pattern A — Queue Spike (fleet sizing problem):
    P50: 120ms, P99: 1,200ms  -> P99/P50 ratio >> 3×
    Signature: queue_depth metric spikes with TTFT P99
    Fix: increase fleet size or improve throughput per GPU

  Pattern B — Long Prompt Starvation (scheduling problem):
    P50: 150ms, P99: 800ms
    Signature: prefill_duration P99 >> P50; correlates with prompt_len P99
    Fix: chunked prefill (see §6.3)

  Pattern C — KV Pressure Admission Delay (memory problem):
    P50: 200ms, P99: 2,000ms
    Signature: kv_utilization > 85% at TTFT P99 spike times
    Fix: KV cache headroom policy (reject or queue at 82% KV util)
```

6.5.2 SLA Budget Allocation Framework

```
SLA BUDGET DESIGN EXAMPLE:
  Target: TTFT P99 < 600ms, TPOT P99 < 60ms

  Budget allocation:
    t_network:     25ms (P99 allowance)
```

```
    t_auth:        15ms (P99 allowance)
    t_queue:       200ms (fleet sized to keep P99 queue ≤ 200ms)
    t_prefill:     250ms (limits max unchunked prompt to ~16K tokens on H100)
    t_first_token: 60ms (= TPOT P99 target)
    Buffer:        50ms (operational headroom)
    TOTAL:         600ms ✓

  Implications of this budget:
    Fleet sizing target: < 75% average GPU utilization
    Chunked prefill: mandatory (to keep t_prefill ≤ 250ms for any prompt)
    TPOT target: 60ms -> batch size must keep TPOT ≤ 60ms
      At 60ms/token: max throughput = 1/0.060 = 16 tokens/sec per sequence
      At batch=32: amortized effectively -> achievable
    Admission control: reject new requests when kv_utilization > 82%
```

6.6 vLLM, TensorRT-LLM, and SGLang — Architecture Comparison

6.6.1 vLLM — Production Open-Source Standard

vLLM (Berkeley RISE Lab, 2023) introduced PagedAttention and continuous batching as an integrated production serving system. It is now the dominant open-source LLM inference runtime.

```
VLLM ARCHITECTURE — KEY COMPONENTS:

  REQUEST LIFECYCLE IN VLLM:

  1. API Server (FastAPI):
     Receives HTTP request, validates, adds to waiting queue.
     Returns streaming response via SSE.

  2. LLMEngine (core scheduler):
     Runs in main process. Executes one "step" per forward pass.
     Each step: selects sequences to run, allocates KV blocks,
     issues work to workers, returns new tokens.

  3. Scheduler:
     Budget-based: tracks num_seqs, num_tokens, KV utilization.
     Admission policy: admit requests from waiting queue when budget allows.
     Preemption: swap or recompute KV if budget exceeded.

  4. PagedAttention KV Cache Manager:
     Divides GPU HBM into fixed-size KV blocks (default: 16 tokens/block).
     Block table: maps (sequence_id, block_idx) -> physical_block_id.
     Eliminates fragmentation (see §6.6.2 below).
     Copy-on-write: multiple sequences can share blocks for prefix caching.

  5. Worker (one per GPU):
     Receives sequence metadata from scheduler.
     Executes GPU forward pass using PagedAttention kernel.
     Returns sampled tokens to scheduler.

  6. Tensor Parallel Workers (multi-GPU):
     All workers execute SAME code on different tensor shards.
     AllReduce after each attention + FFN via NCCL.
     Synchronized by scheduler at each step boundary.

VLLM KEY CONFIGURATION PARAMETERS:

  --model:                  Model path or HuggingFace ID
  --tensor-parallel-size:  TP degree (e.g., 8 for 8-GPU node)
  --gpu-memory-utilization: Fraction of GPU HBM for KV cache (default: 0.90)
                            Total KV pool = GPU_HBM × util - model_weights
  --max-num-seqs:           Max concurrent sequences (default: 256)
  --max-model-len:          Maximum sequence length supported
  --block-size:             KV block size in tokens (default: 16)
```

```
  --enable-prefix-caching: Enable radix tree prefix sharing
  --enable-chunked-prefill: Enable chunked prefill scheduling
  --max-num-batched-tokens: Chunk size for chunked prefill (default: 512)
  --quantization:          "fp8", "awq", "gptq", "squeezellm"
  --dtype:                 "bfloat16", "float16", "float32"

  KV POOL CALCULATION:
    Total HBM: 80 GB (H100) × 8 GPUs = 640 GB
    Model (LLaMA-3 70B FP8): 70 GB
    KV pool = (640 × 0.90) - 70 = 506 GB
    At BF16 KV, LLaMA-3 70B GQA, S=4096:
      1.34 GB/seq → 506 / 1.34 = ~378 max concurrent sequences
    At FP8 KV: 0.67 GB/seq → ~755 max concurrent sequences
```

6.6.2 PagedAttention — The KV Memory Breakthrough

```
THE FRAGMENTATION PROBLEM (pre-PagedAttention):

  Old approach: pre-allocate max_seq_len × KV_per_token bytes per request.
  For max_seq_len=4096, KV=0.16 MB/token:
    Reserved per request: 4096 × 0.16 MB = 655 MB
    Actual usage (avg 800-token requests): 128 MB
    Waste per request: (655-128)/655 = 80% of KV allocation wasted!

  At batch=20:
    HBM reserved: 20 × 655 MB = 13.1 GB
    HBM actually used: 20 × 128 MB = 2.56 GB
    80% of KV HBM budget wasted on internal fragmentation.

PAGEATTENTION SOLUTION — OS PAGING FOR KV CACHE:

  Allocate KV cache in fixed-size blocks ("pages"), 16 tokens per block.
  Block table maps (seq_id, logical_block) -> physical_block_id.
  Physical blocks do NOT need to be contiguous.

  Sequence of 50 tokens = 4 physical blocks (ceil(50/16) = 4 blocks).
  Each block allocated on-demand as the sequence grows.
  On sequence completion: all blocks immediately freed and reused.

  Block table example (one sequence):
    Logical block 0 -> Physical block #47
    Logical block 1 -> Physical block #12
    Logical block 2 -> Physical block #103
```

```
    (no contiguity required — attention kernel handles indirection)

  FRAGMENTATION:
    Internal fragmentation per sequence:
      Only the LAST block is partially filled.
      Expected waste = (block_size - 1) / 2 = 7.5 tokens per sequence.
      As fraction of sequence: 7.5 / avg_length
      For avg_length=800: 7.5/800 = 0.9% waste (vs 80% before!)

  SHARING — Copy-on-Write KV Blocks:
    Multiple sequences with the same prefix can SHARE KV blocks.
    System prompt (1K tokens): one set of blocks, shared by all requests.
    When a sequence diverges: copy-on-write allocates a new physical block.
    Effective KV reduction for chatbot: 50-80% fewer KV blocks allocated.

  THROUGHPUT IMPACT:
    PagedAttention vs contiguous allocation:
      Batch size sustainable at same memory: 2-4× higher
      Throughput improvement: 2-4× at similar latency
      Gains larger for: longer sequences, more variable lengths
```

6.6.3 TensorRT-LLM — NVIDIA's Production Stack

```
TRT-LLM ARCHITECTURE — BUILD-TIME vs RUNTIME:

  TRT-LLM separates optimization into two phases:

  BUILD TIME (offline, one-time per model/hardware combination):
    Input: Model weights + target GPU + batch range + sequence length range
    Output: Serialized .engine file (GPU-specific optimized binary)

    Build operations:
      Graph optimization:    dead code, constant folding, layer fusion
      Kernel specialization: GEMM shapes pre-computed, cuBLAS workspace sized
      INT8/FP8 calibration:  if quantization requested, calibrate at build
      CUDA Graph capture:    decode-path graphs captured at fixed batch sizes
      Plugin registration:   TRT-LLM custom plugins (FlashAttention, etc.)

    Build time: 5-30 minutes per configuration.
    Must rebuild when: GPU changes, quantization changes, max_seq_len changes.

  RUNTIME (deployment, per-request serving):
    Load .engine file, execute with TRT runtime.
```

```
    CUDA Graphs replay for decode (zero CPU overhead per decode step).
    Inflight batching (TRT-LLM's continuous batching implementation).
    In-flight FP8 quantization (no model rebuild for FP8 serving).

KEY ADVANTAGE vs VLLM:
  CUDA Graph capture eliminates CPU dispatch overhead for decode:
    Without CUDA Graphs: ~1ms CPU overhead per decode step
    With CUDA Graphs: ~0.01ms overhead per decode step
    At 100 decode steps, batch=32: saves 100 × 1ms × 32 = 3.2s CPU time
    This is why TRT-LLM achieves 15-25% lower TPOT vs vLLM for same hardware.

WHEN TO USE TRT-LLM vs VLLM:
  Use TRT-LLM for:
    Maximum throughput production deployment
    NVIDIA hardware (H100, B200) — deeply optimized for NVIDIA TCs
    Stable model configuration (rebuild time is a cost)
    Latency-sensitive serving where every ms matters

  Use vLLM for:
    Rapid iteration and model experimentation
    AMD GPU support (ROCm backend)
    Python-native integration for custom sampling/constraints
    Multi-modal models, diverse model architectures
    Simpler deployment and operations
```

6.6.4 SGLang — Structured Generation Efficiency

```
SGLANG (Stanford, 2024) — KEY DIFFERENTIATOR:

  Designed for structured generation and multi-call programs.
  Key innovation: RadixAttention (enhanced prefix caching).

  RADIXATTENTION vs VLLM PREFIX CACHING:
    vLLM: prefix cache based on exact string match of token IDs.
    SGLang RadixAttention: radix tree structure that shares prefixes
      across calls in a PROGRAM, not just across separate requests.

  Use case: multi-turn agents, chain-of-thought, tool calls.
  Example program with 5 LLM calls sharing a 2K system prompt:
    vLLM: 5 × 2K prefill = 10K tokens of KV computed
    SGLang: 1 × 2K prefill + 4 × cache hit = 2K tokens + 4 cache lookups
    Improvement: 5× prefill reduction for program-level inference.
```

```
WHEN SGLANG WINS:
  Agent frameworks (LangChain, LlamaIndex workloads)
  Multi-call LLM programs with shared context
  RAG systems with fixed retrieval context
  Programmatic constrained generation (JSON, code)

THROUGHPUT vs VLLM (program-level workloads):
  2-3× better throughput for agent workloads (high prefix reuse)
  Comparable to vLLM for single-request workloads
```

6.7 Speculative Decoding at the Serving Layer

Speculative decoding exploits the observation that many decode steps are "easy" — the model is highly confident. A small draft model proposes several tokens; the target model verifies all of them in one parallel forward pass.

```
SPECULATIVE DECODING MECHANISM:

  Step 1: Draft model (7B) proposes K=4 candidate tokens in sequence.
          Time: 4 × TPOT_7B (cheap, fast GEMV)

  Step 2: Target model (70B) verifies all 4 drafts in ONE forward pass.
          The target processes position N with token draft_1,
          position N+1 with draft_2, etc. — all in parallel.
          Time: 1 × TPOT_70B (same cost as a single step)

  Step 3: Acceptance:
          If target agrees with draft_k: accept, continue
          If target rejects draft_k: resample from target at position k,
          discard draft tokens k+1 onwards.

  Expected accepted tokens per step = K × acceptance_rate
  Expected speedup = (1 + K × alpha) / (1 + overhead_ratio)

  where:
    K = draft tokens per step (typically 4-8)
    alpha = acceptance rate per token (depends on draft/target similarity)
    overhead_ratio = draft_model_cost / target_model_cost

  WHEN SPECULATIVE DECODING HELPS:
    acceptance_rate > 0.70:  Strong speedup (2-3×)
    acceptance_rate 0.50-0.70: Moderate speedup (1.5-2×)
    acceptance_rate < 0.50:  Overhead dominates, slower than baseline

  TYPICAL ACCEPTANCE RATES:
    Llama-3 8B drafting Llama-3 70B: alpha ≈ 0.65-0.80 (same family)
    Mistral 7B drafting Llama-3 70B: alpha ≈ 0.40-0.55 (cross-family)
    Codestral 22B drafting DeepSeek Coder 236B: alpha ≈ 0.75+ (code tasks)

  EXPECTED SPEEDUP — LLaMA FAMILY (draft=8B, target=70B, K=4, alpha=0.75):
    Expected tokens per step = 4 × 0.75 = 3.0 tokens
    vs baseline: 1.0 token per step
    Raw speedup: 3.0×
    After overhead (8B GEMV cost ≈ 8% of 70B): ~2.5-2.8×
    With batched verification: ~2-3× TPOT improvement

  MEMORY COST:
    Draft model must fit in GPU memory alongside target.
    LLaMA-3 8B FP8: 8 GB additional HBM required.
```

```
    At H100 80 GB with 70 GB for weights: 80-70-8 = 2 GB for KV (tight!)
    Practical: use H200 (141 GB) or MI300X (192 GB) for production spec-decode.

  WHEN NOT TO USE SPECULATIVE DECODING:
    High batch sizes (batch > 32): target model is already compute-bound.
      Adding draft overhead without proportional benefit.
    Highly creative / diverse outputs: alpha drops below 0.50.
    Memory-constrained deployments: draft model consumes KV budget.
    Real-time batch API: overhead per step may not be amortized.
```

6.8 Disaggregated Prefill and Decode

6.8.1 Motivation — Hardware Mismatch Between Phases

```
THE HARDWARE MISMATCH PROBLEM:

  Prefill is COMPUTE-BOUND -> optimal hardware: high TFLOPS (H100 SXM)
  Decode is BANDWIDTH-BOUND -> optimal hardware: high HBM BW (H200, MI300X)

  In collocated serving (prefill + decode on same GPU):
    During prefill: GPU optimized for compute (good)
    During decode: GPU used for bandwidth (good)
    Problem: GPU switches between phases, never fully optimized for either.

    H100 at prefill: using 989 TFLOPS efficiently
    H100 at decode: using 3.35 TB/s bandwidth, leaving TFLOPS idle
    Mixed workload: GPU never fully in either optimal state.
    Utilization: ~40-55% of theoretical peak for mixed workload.

DISAGGREGATED SOLUTION (Splitwise / Mooncake / NVIDIA Dynamo):
  Prefill cluster (compute-optimized): H100 SXM or A100 SXM
    Runs ONLY prefill. No decode. Always in compute-bound state.
  Decode cluster (bandwidth-optimized): H200 or MI300X
    Runs ONLY decode. No prefill. Always in bandwidth-bound state.

  KV transfer: after prefill, copy KV cache from prefill GPU to decode GPU.
    Transfer size: 1.34 GB per request (LLaMA-3 70B, 4K context, BF16 KV)
    Transfer time: 1.34 GB / 50 GB/s (IB NDR) = 26.8ms
    Transfer time: 1.34 GB / 900 GB/s (NVLink, same node) = 1.5ms
```

6.8.2 Disaggregation Tradeoffs

```
DISAGGREGATION QUANTIFIED (LLaMA-3 70B, H100/H200 mix):

  COLLOCATED (8× H100 TP=8):
    Cost/1M tokens: $X (baseline)
    TTFT P99: ~180ms
    Throughput: ~6,500 tok/s

  DISAGGREGATED (2× H100 prefill + 8× H200 decode):
    Cost/1M tokens: ~$0.68X  (32% cheaper)
    TTFT P99: ~195ms (+15ms KV transfer, acceptable)
    Throughput: ~9,200 tok/s (+41%)
    Break-even fleet size: ~120 H-class GPU equivalents

WHEN TO DISAGGREGATE:
  Scale > 100 GPUs: complexity amortized across large fleet
  Average prompt length > 1,024 tokens: prefill cost is significant
  Average output length > 500 tokens: decode dominates, worth optimizing
  TTFT SLA > 300ms: room for +15-25ms KV transfer penalty
  Cost optimization is top priority (batch/API workloads)

WHEN TO STAY COLLOCATED:
  Scale < 50 GPUs: operational complexity not worth it
  Short prompts + short outputs: transfer overhead dominates benefit
  TTFT SLA < 100ms: no room for transfer latency
  Real-time interactive with strict latency SLO
  Engineering team lacks bandwidth for multi-cluster ops
```

6.9 Multi-GPU Inference — Tensor Parallelism for Serving

```
TENSOR PARALLELISM FOR SERVING — WHEN AND HOW:

  Model too large for single GPU:
    LLaMA-3 70B BF16: 140 GB -> doesn't fit H100 (80 GB)
    Minimum: TP=2 (2× H100 SXM, 160 GB total, model fits + KV headroom)
    Better: 1× H200 (141 GB) or 1× MI300X (192 GB) — eliminates TP entirely

  LATENCY BENEFIT FROM TP (even when model fits):
    TP distributes compute across GPUs.
    For TPOT: TP=4 roughly halves TPOT vs TP=2 (if NVLink bandwidth allows).
```

```
    TP=1, LLaMA-3 70B FP8, H200:        TPOT P50 ≈ 35ms
    TP=4, LLaMA-3 70B FP8, 4×H200:      TPOT P50 ≈ 12ms (2.9×)
    TP=8, LLaMA-3 70B FP8, 8×H200:      TPOT P50 ≈ 8ms (4.4×)
    Diminishing returns at TP=8 vs TP=4: AllReduce overhead grows.

  TP COMMUNICATION OVERHEAD FOR SERVING:
    Per decode step: 2 AllReduces per transformer layer × L=80 layers = 160
AllReduces
    At NVLink (900 GB/s), message size ~67 MB per layer:
      Per AllReduce: 0.26 ms (negligible)
      Total overhead: 160 × 0.26 = 42ms per step (too high at TP=8!)

    WAIT — this seems wrong. Let's recalculate for serving:
    At batch=1, decode step (single token):
      Activation size per AllReduce: 1 × d_model × 2B = 1 × 8192 × 2 = 16 KB
      (much smaller than training — only 1 token per step)
      Per AllReduce: 16 KB / 900 GB/s = 0.018 μs → negligible

    AllReduce overhead for serving (batch=1): < 0.1 ms total per step.
    AllReduce overhead for serving (batch=32): ~32 × 16 KB = 512 KB
      512 KB / 900 GB/s = 0.57 μs → still negligible vs 35ms TPOT.

    CONCLUSION FOR SERVING:
      TP within NVLink domain: AllReduce overhead is negligible.
      TP across InfiniBand (different nodes): 2-5 μs per AllReduce.
      At 160 AllReduces per step: 160 × 5μs = 0.8ms overhead per step.
      At TPOT=35ms: 0.8ms = 2.3% overhead — acceptable but non-trivial.

TP SELECTION GUIDE FOR SERVING:
  TP=1:  Use when model fits on a single GPU AND TPOT SLA is met.
         Best TCO (one GPU per serving instance).
  TP=2:  Use when model requires 2 GPUs OR TPOT needs halving.
         Ideal: 1× H200 pair within NVLink.
  TP=4:  Use when TPOT SLA < 15ms requires more bandwidth.
         Must stay within one NVLink domain (8-GPU DGX).
  TP=8:  Maximum practical; diminishing returns from AllReduce fraction.
         Only for models > 160 GB (BF16) or extreme TPOT SLAs (< 8ms).
```

6.10 Observability Hooks for Production Inference

```
CRITICAL SERVING METRICS AND THEIR ALERT THRESHOLDS:

  Metric                    Type    Target         Critical   Source
  ──────────────────────────────────────────────────────────────────────
  TTFT P99                  SLA     < SLA target   > SLA×1.5  App metrics
  TPOT P99                  SLA     < SLA target   > SLA×1.5  App metrics
  GPU Utilization           Health  70-80%         < 40%      DCGM
  KV Cache Utilization      Health  < 82%          > 87%      vLLM metrics
  Queue Depth               Health  < 5 requests   > 20       App metrics
  Prefill Throughput        Perf    > 10K tok/s    < 5K tok/s App metrics
  Decode Throughput         Perf    > 2K tok/s     < 1K tok/s App metrics
  Token Sampling Latency    Perf    < 1ms          > 5ms      App metrics
  Request Success Rate      SLA     > 99.9%        < 99%      App metrics
  GPU Temperature           Health  < 78°C         > 82°C     DCGM
  NVLink Bandwidth          Health  > 800 GB/s     < 600 GB/s DCGM
  ──────────────────────────────────────────────────────────────────────


VLLM PROMETHEUS ENDPOINT:
  vLLM exposes metrics at /metrics by default.
  Key counters:
    vllm:num_requests_running:       Current active decode sequences
    vllm:num_requests_waiting:       Queue depth
    vllm:gpu_cache_usage_perc:       KV cache utilization fraction
    vllm:num_preemptions_total:      KV evictions (should be near 0)
    vllm:time_to_first_token_seconds: TTFT histogram
    vllm:time_per_output_token_seconds: TPOT histogram
    vllm:request_prompt_tokens:      Prompt length distribution
    vllm:request_generation_tokens:  Output length distribution

DCGM METRICS FOR SERVING HEALTH:
  dcgmi dmon -e 100,155,1003,1004,1005,1009
  Field IDs:
    100:  GPU temperature (°C)
    155:  Power draw (W)
    1003: SM utilization (%)
    1004: Memory utilization (%)
    1005: HBM bandwidth utilization (%)
    1009: NVLink bandwidth (bytes/sec)
```

6.11 Hardware Selection for Inference Workloads

```
HARDWARE SELECTION DECISION TABLE — LLM INFERENCE:

  Workload                      Recommended GPU     Reason
  ──────────────────────────────────────────────────────────────────────
  7B-13B BF16, any scale        H100 SXM or H200   Both fit, H200 better BW
  70B BF16, TTFT-sensitive       H200 or MI300X    Single-GPU serving (no TP)
  70B BF16, throughput-first    MI300X or 2×H100   Single GPU vs TP=2
  70B FP8, high throughput      H100 SXM TP=2      Best TFLOPS for FP8 GEMM
  70B INT4, cost-sensitive      H100 PCIe or A100  Model fits on single card
  405B BF16, production         5×MI300X or 12×H100 Memory capacity required
  Short latency (TPOT<10ms)     4×H200 TP=4        NVLink bandwidth needed
  Batch API, throughput         H100×8 TP=8        Max throughput, cost OK
  Mixed chatbot fleet           H200                Best across workload types
  ──────────────────────────────────────────────────────────────────────


COST PER TOKEN COMPARISON (LLaMA-3 70B FP8, owned hardware):

  Config              Tok/s   GPU Cost/hr  Cost/1M tok   vs Baseline
  ──────────────────────────────────────────────────────────────────
  2×H100 TP=2         4,400   $3.32        $0.209        baseline
  1×H200              3,100   $1.80        $0.161        -23%
  1×MI300X            2,800   $1.50        $0.149        -29%
  8×H100 TP=8         6,800   $13.28       $0.542        +160%  (latency, not
cost)
  ──────────────────────────────────────────────────────────────────


  KEY INSIGHT: Higher GPU count reduces latency but increases cost per token.
  Single-GPU serving (H200, MI300X) is the cost-optimal path for 70B.
  Multi-GPU (TP=8) is for TPOT SLA that single-GPU cannot meet.
```

6.12 Multi-Tenant Serving Cost Model [r1]

6.12.1 The Cost-Per-Token Formula

```
COST-PER-TOKEN — FULLY LOADED CALCULATION:

  cost_per_token = GPU_cost_per_hour / (tokens_per_second × 3600)

GPU COST REFERENCE (2026): [ESTIMATED]
```

```
  Owned H100 SXM5 (3-year depreciation):
    Hardware: ~$30,000 / (3 × 8,760h) = $1.14/GPU-hr
    Power (700W × $0.07/kWh × PUE 1.10): $0.054/GPU-hr
    Cooling + datacenter overhead: $0.35/GPU-hr
    Ops + maintenance: $0.12/GPU-hr
    TOTAL OWNED H100: ~$1.66/GPU-hr (fully loaded)

  Cloud H100 (on-demand): $3.50-4.50/GPU-hr
  Cloud H100 (1-year reserved): $2.50-3.50/GPU-hr
  Cloud H100 (spot/preemptible): $1.20-1.80/GPU-hr

WORKED EXAMPLE — LLaMA-3 70B PRODUCTION SERVING:
  Config: 1× H100 SXM5, FP8 weights, vLLM, continuous batching
  Achieved throughput: 3,200 tok/s at batch=32 (decode phase)

  At 100% utilization:
    cost_per_token = $1.66 / (3,200 × 3,600) = $0.000144 = $0.14/1K tokens

  At 40% utilization (realistic production average):
    Effective tok/s = 3,200 × 0.40 = 1,280 tok/s
    cost_per_token = $1.66 / (1,280 × 3,600) = $0.00036 = $0.36/1K tokens

  LESSON: GPU utilization is the single biggest lever on cost per token.
  Going from 40% to 70% utilization: cost/token drops from $0.36 to $0.21.
  This alone is a 42% cost reduction with no hardware change.
```

6.12.2 The Utilization-Latency Tradeoff (Queueing Theory)

```
LITTLE'S LAW FOR INFERENCE SERVING:
  L = λ × W
  L = mean requests in system
  λ = arrival rate (req/s)
  W = mean time in system (queue + inference)

  M/M/1 queue approximation:
  Mean queue depth at utilization ρ: E[N_queue] = ρ² / (1 - ρ)

  GPU Utilization   Queue Depth   P99 TTFT Multiplier
  ──────────────────────────────────────────────────
  50%               0.50          ~1.4×
  70%               1.63          ~2.1×
  80%               3.20          ~3.5×
  85%               4.82          ~5.0×
```

```
  90%                8.10          ~9.0×  <- SLA breach zone
  95%                18.05         ~20×   <- collapse
  ──────────────────────────────────────────────────────

  PRACTICAL OPERATING RANGE:
    Interactive serving (chat): 65-75% GPU utilization target
    Batch API serving:          75-85% GPU utilization target
    Never exceed:               82% sustained (P99 enters collapse zone)

  ADMISSION CONTROL RULE:
    When KV utilization > 82%: reject new requests with 429 (Too Many Requests)
    Or queue with timeout: if queue_depth > max_queue_size, return 503.
    Better to return a clean error than degrade everyone's TTFT.
```

6.12.3 Multi-Tenant Cost Architecture

```
MULTI-TENANT SERVING DESIGN:

  SHARED FLEET (maximize utilization):
    Single GPU fleet serves all tenant tiers.
    Scheduler maintains priority queues per SLA tier.
    KV cache split: premium tier has reserved blocks.

  Tier design example:
    Premium (TTFT < 200ms):  Priority=1, 30% KV budget reserved
    Standard (TTFT < 800ms): Priority=2, 40% KV budget shared
    Batch (TTFT < 5s):       Priority=3, uses spare capacity

  Revenue optimization:
    Premium users pay 5-10× per token vs batch.
    Same GPU fleet serves both — premium users fill latency-sensitive slots,
    batch users fill off-peak capacity.
    Fleet utilization target: 80%+ with priority queuing.

  ISOLATION (compliance / enterprise):
    Dedicated GPU pools per tenant.
    Prevents cross-tenant KV cache side-channels.
    Lower utilization (each pool runs at ~50% avg).
    Justified for: regulated industries, enterprise SLA guarantees.
```

6.13 The Tokens-Per-Second Optimization Hierarchy [r1]

When asked to maximize tokens/second, work through these levers in order. Each has a prerequisite and a ceiling. Applying them out of order wastes engineering time.

TOKENS-PER-SECOND OPTIMIZATION HIERARCHY:

LEVER	MECHANISM	TYPICAL GAIN	PREREQUISITE
1. Continuous batching	Fill GPU constantly (remove padding waste)	2-4×	Always do first. No excuses.
2. Quantization (FP8/INT8/INT4)	FP8: halve weight bytes loaded	2× decode 4× INT4	Quality calibration. Accept 0.3-3% quality. Match to workload type.
3. KV cache runtime. optimization	PagedAttention: eliminate fragment. GQA: 8× KV reduction	2-4× more concurrency	Modern serving model? GQA already in INT8 KV: halve KV size.
4. Parallelism tuning	TP within NVLink: lower TPOT PP for large models	Linear with GPU count	NVLink domain. Won't help throughput (helps latency only).
5. Flash Attention (already on)	IO-aware tiling: O(S) HBM traffic	1.5-3× on attention	Built into vLLM/TRT. Just verify it's active.
6. CUDA Graphs (decode path)	Eliminate CPU dispatch overhead	15-25% TPOT	TRT-LLM or vLLM with graph mode enabled.
7. Speculative decoding	2-3 tokens per target step	2-3× on low batch	Small draft model fits in memory. High alpha needed.
8. Disaggregated P-D	Prefill/decode separate hardware	30-50% utilization	> 100 GPU fleet. Operational complexity.

CRITICAL TRADEOFF TO STATE EXPLICITLY:

Maximizing tokens/second (throughput) DIRECTLY CONFLICTS with

```
    minimizing TTFT and TPOT (latency).

    Large batches -> higher throughput, higher TTFT (more queue time)
    Small batches -> lower TTFT, lower throughput, higher cost/token

    Always clarify the service objective before proposing optimizations.
    "Are we optimizing for throughput (batch API) or latency (interactive chat)?"
    The answer changes the entire recommendation.

OPTIMIZATION IMPACT TABLE (LLaMA-3 70B, H100 SXM5, batch=32):

  Config                        Throughput   TPOT P50  Cost/1M tok   Notes
  ──────────────────────────────────────────────────────────────────────────
  BF16, static batching         1,200 tok/s  85ms      $0.38         Pre-2022
baseline
  BF16, continuous batching     3,600 tok/s  35ms      $0.13         +200%
throughput
  FP8, cont. batching           5,800 tok/s  22ms      $0.079        +61% vs BF16 CB
  FP8 + INT8 KV                 6,400 tok/s  20ms      $0.072        +10% more
concurrency
  FP8 + INT8 KV + spec-dec      8,200 tok/s  14ms      $0.056        +28% (batch=8-
16)
  ──────────────────────────────────────────────────────────────────────────
  Total improvement:            6.8× vs baseline at 32% of baseline cost/token
```

6.14 Key Takeaways and Review Questions

```
NUMBERS TO MEMORIZE COLD:

  Maximum decode throughput (LLaMA-3 70B BF16, batch=1):
    H100: 3350/140 × 0.70 = ~17 tok/s
    H200: 4800/140 × 0.70 = ~24 tok/s
    MI300X: 5300/140 × 0.70 = ~27 tok/s

  Prefill regime: COMPUTE-BOUND (AI ~ 2,000 FLOP/byte at N=2048)
  Decode regime:  MEMORY-BOUND (AI ~ 1 FLOP/byte at batch=1)
  Crossover batch size (H100, 70B BF16): ~148 tokens

  Continuous batching gain: 2-4× throughput vs static batching
  Chunked prefill: eliminates TTFT P99 spikes from long prompts
  PagedAttention: reduces KV fragmentation from 60-80% to < 1%
```

```
  Fleet utilization target: 65-80% for interactive, never > 82% sustained

  Cost fully loaded (owned H100): ~$1.66/GPU-hr
  Cost per 1M tokens (H100 BF16, 3,200 tok/s, 70% util): ~$0.21

KEY PRINCIPLES:

  1. PREFILL IS COMPUTE-BOUND, DECODE IS MEMORY-BOUND.
     These require different optimizations. Treating them the same
     is the most common mistake in inference engineering.
     More TFLOPS helps prefill, not decode.
     More HBM bandwidth helps decode, not prefill.

  2. CONTINUOUS BATCHING IS NON-NEGOTIABLE.
     Static batching wastes 50-70% of GPU time.
     Enable continuous batching before any other optimization.
     It is the single highest-ROI serving optimization.

  3. KV CACHE MANAGEMENT IS THE SERVING BOTTLENECK.
     At scale, the constraint is almost always KV HBM capacity, not compute.
     GQA + PagedAttention + INT8 KV = the three-part KV management stack.
     Combined: 16× more concurrent users than naive contiguous BF16 KV.

  4. UTILIZATION IS THE COST LEVER, NOT THROUGHPUT.
     Cost per token scales inversely with GPU utilization.
     70% utilization vs 40% utilization: 42% cost reduction, same hardware.
     Fleet sizing drives cost more than any kernel optimization.

  5. LATENCY AND THROUGHPUT CONFLICT — ALWAYS NAME THE TRADEOFF.
     High batch = high throughput, high TTFT.
     Low batch = low TTFT, low throughput.
     Chunked prefill partially resolves this for interactive workloads.
     Principal engineers name this conflict explicitly before proposing solutions.
```

Review Questions

Difficulty: Foundational

Question 1 (Prefill vs Decode Regime): Classify and justify the performance regime for each scenario using the roofline model. (a) LLaMA-3 70B BF16, prefill with N=4096 prompt tokens on H100. (b) LLaMA-3 70B BF16, decode with batch=256 on H100. (c) LLaMA-3 70B INT4 (W4A16), decode with batch=1 on H100.

Expected answer: (a) FLOPs = 2 × 70B × 4096 = 574 TFLOPS. Bytes = 140 GB. AI = 574e12/140e9 = 4,100 FLOP/byte >> 295. Compute-bound ✓. H100 Tensor Cores are the bottleneck. Optimization: FlashAttention (reduces attention FLOPs at long S), FP8 (2× throughput for GEMMs). (b) AI ≈ 2 × batch_size = 2 × 256 = 512 FLOP/byte > 295. Compute-bound ✓ (the batch is large enough to cross the ridge point). H100 at ~800-900 TFLOPS effective. Optimization: FP8 GEMM, larger tile sizes. (c) INT4 model = 70B × 0.5 bytes = 35 GB. FLOPs = 2 × 70B × 1 = 140 GFLOPs. AI = 140e9 / 35e9 = 4 FLOP/byte. Memory-bound ✓ (INT4 helps by reducing bytes, but at batch=1 still deeply memory-bound). Decode throughput = 3,350 GB/s / 35 GB = 95 tok/s theoretical, ~67 tok/s practical — 4× faster than BF16 purely from bandwidth reduction.

Difficulty: Intermediate

Question 2 (Continuous Batching Quantification): A LLaMA-3 70B service is running static batching with batch=8. Request output lengths follow a bimodal distribution: 50% of requests are 20 tokens, 50% are 200 tokens. (a) What fraction of GPU decode time is wasted on padding in a batch of 8? (b) If continuous batching were enabled, estimate the throughput improvement. (c) A colleague suggests "just set batch=8 and all requests will finish at similar times since the average is 110 tokens." Explain the flaw in this reasoning.

Expected answer: (a) In a batch of 8 with bimodal distribution: ~4 requests will be 20-token, ~4 will be 200-token. GPU runs 200 decode steps for the full batch. For the 4 short requests (20 tokens): they complete at step 20. Steps 21-200 = 180 steps × 4 slots = 720 slot-steps of wasted compute. Total slot-steps = 200 × 8 = 1,600. Waste fraction = 720/1,600 = 45% of GPU decode time wasted on finished sequences. (b) Continuous batching eliminates this waste. As each 20-token request finishes at step 20, a new request immediately fills the slot. Effective batch stays near 8 throughout. Throughput improvement: roughly 1/(1-0.45) = 1.82× (80% utilization vs 55%). In practice: 1.5-2× for this distribution. (c) Flaw: averaging obscures the distribution. The GPU must run until the LONGEST request in the batch completes — 200 steps, not 110 steps. Half the slots sit idle for 90 steps. Batch efficiency is determined by the tail of the output distribution, not the mean. This is Jensen's inequality applied to batching — E[max(outputs)] >> max(E[outputs]) when the distribution has high variance.

Difficulty: Principal Level

Question 3 (System Design — Serving 100M Users): Design the inference serving system for a ChatGPT-scale service: 100M DAU, 50,000 peak RPS, average 2,000 input tokens, average 500 output tokens, TTFT P99 < 800ms, TPOT P99 < 60ms. Model: LLaMA-3 70B equivalent, H100 SXM5. (a) Fleet size calculation. (b) KV cache sizing per node. (c) Batching strategy selection with justification. (d) One optimization that halves cost without violating SLAs.

Expected answer: (a) Fleet sizing: avg latency = TTFT + output×TPOT = 250ms + 500×25ms = 12.75s. N_GPUs = (RPS × avg_latency) / target_utilization. Each H100×8 TP node delivers ~6,500 tok/s. Throughput needed: 50,000 RPS × 500 output tokens = 25M tok/s. At 70% utilization: need 25M / (0.70 × 6,500) = 5,495 GPU-nodes (8 GPU each) = 43,960 GPUs. Rounded: ~5,500 H100 8-GPU nodes (44,000 GPUs). (b) Per 8-GPU TP node: KV headroom = (80 GB × 8) - 70 GB weights = 570 GB. With FP8 KV (0.67 GB per 4K-context seq): 570/0.67 = 850 concurrent sequences per node. Matches throughput needs well. (c) Batching strategy: continuous batching (mandatory for this scale), chunked prefill (mandatory — 2K avg input means long prompts will occur; need to protect TTFT P99), priority queue with two tiers (interactive: TTFT priority; batch API: throughput priority). max_num_seqs = 256, chunk_size = 512 tokens, KV admission control at 82% utilization. (d) Best single optimization: FP8 weights + FP8 KV cache. FP8 weights (70 GB) leave 570 GB for KV on same 8-GPU node. FP8 KV halves KV size to 0.335 GB/seq → 1,700 concurrent sequences per node (2× more). This means 2× fewer nodes needed at same throughput = $22M/month savings (assuming $2/GPU-hr, 44,000 GPUs). Quality cost: <0.3% on standard benchmarks. This is unambiguously the highest-ROI intervention.

Difficulty: Principal Level

Question 4 (TTFT Diagnosis): Your production LLaMA-3 70B serving cluster shows: TTFT P50=180ms, TTFT P99=1,800ms. GPU utilization averages 71%. TPOT P50=28ms (within SLA). Chunked prefill is enabled with chunk_size=512. KV utilization averages 68% but spikes to 91% for 5-minute windows every 2 hours. Diagnose the root cause and prescribe the fix.

Expected answer: The P99/P50 TTFT ratio of 10× combined with 71% average utilization and periodic KV spikes to 91% points to Pattern C: KV pressure admission delay. This is not a queue depth problem (70% utilization should be safe) or a prefill monopoly problem (chunked prefill is enabled). The signature: every 2 hours, traffic bursts push KV utilization to 91%. At 91% KV utilization, the scheduler cannot admit new requests — they queue until KV pressure drops. During this 5-minute window, queue depth spikes → TTFT P99 at 1,800ms. Diagnosis confirmed: P99 timing correlates with KV spike windows (check timestamps). Fix: (1) Immediate: lower KV admission threshold from default (typically 90%) to 82%. When KV > 82%, return 429 instead of queuing. This bounds P99 TTFT at the cost of some request rejections. (2) Medium-term: enable INT8 KV cache. Halves KV size per sequence → same HBM supports 2× more concurrent sequences → KV peaks drop from 91% to ~45% at same traffic. (3) Capacity: if the traffic burst is genuine demand, add GPUs during those 2-hour windows (auto-scaling trigger on KV utilization > 75%).

Further Reading

1. *Efficient Memory Management for Large Language Model Serving with PagedAttention* —

Kwon et al. (2023). `arxiv.org/abs/2309.06180` The foundational vLLM paper. Read the memory waste analysis and block table design.

2. *Orca: A Distributed Serving System for Transformer-Based Generative Models* — Yu et al. (2022). OSDI 2022. Continuous batching (iteration-level scheduling): the single most important LLM serving innovation. Read the scheduling algorithm section.
3. *Fast Distributed Inference Serving for Large Language Models* — Sheng et al. (2023). `arxiv.org/abs/2305.05920` FlexGen and memory hierarchy trading; complementary to vLLM.
4. *Splitwise: Efficient Generative LLM Inference Using Phase Splitting* — Patel et al. (2024). `arxiv.org/abs/2311.18677` The foundational disaggregated prefill-decode paper. Read the utilization analysis.
5. *Sarathi-Serve: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills* — Agrawal et al. (2024). `arxiv.org/abs/2308.16369` The chunked prefill paper. The TTFT spike analysis is the clearest treatment available.
6. *NVIDIA TensorRT-LLM Documentation* — NVIDIA (2024). `github.com/NVIDIA/TensorRT-LLM` The TRT-LLM architecture guide. Read the inflight batching and CUDA Graph sections.

— End of Chapter 6 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved